

Day 2：前端技术复习与强化 ✓>

*☐今日时间安排

时段	时间	内容	形式
上午	9:00-10:30	HTML5语义化与现代CSS	理论+实践
	10:45-12:00	JavaScript ES6+核心特性	动手编码
下午	14:00-15:30	Vue3基础与Composition API	框架学习
	15:45-17:00	组件通信与状态管理	项目实战
晚上	19:00-20:30	Vue Router与项目整合	完整应用
	20:45-21:00	项目验收与总结	自测

✓✓ 学习目标

☐今日完成后，你将能够：

✓ 编写语义化HTML - 使用HTML5标签构建可访问性良好的页面结构 ✓ 掌握现代CSS布局 - 熟练使用Flexbox、Grid、响应式设计 ✓ 运用ES6+特性 - 使用箭头函数、解构赋值、Promise/async-await等 ✓ 精通Vue3 Composition API - 用setup语法糖组织组件逻辑 ✓ 实现状态管理 - 使用Pinia进行全局状态管理 ✓ 构建SPA应用 - 使用Vue Router实现单页应用路由

** 详细学习内容

☐1. HTML5 语义化结构（1.5小时）

☐1.1 为什么需要语义化？

传统div soup的问题：

```
<!-- ❌差：全是div，难以理解结构 -->
<div class="header">
  <div class="nav">
    <div class="logo">...</div>
    <div class="menu">...</div>
  </div>
</div>
<div class="main">
  <div class="article">...</div>
  <div class="sidebar">...</div>
</div>
<div class="footer">...</div>
```

语义化标签的优势：

```
<!-- ✅好：语义清晰，可访问性好 -->
<header>
  <nav>
    <a class="logo" href="/">...</a>
    <ul>...</ul>
  </nav>
</header>
<main>
  <article>...</article>
  <aside>...</aside>
</main>
<footer>...</footer>
```

✓ SEO友好 - 搜索引擎更容易理解页面结构 ✓ 可访问性提升 - 屏幕阅读器能正确朗读内容 ✓ 代码可维护 - 开发者容易理解页面意图 ✓ 样式更灵活 - 减少class依赖

❑1.2 核心语义化标签

```
<!DOCTYPE html>
<html lang="zh-CN">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>页面标题</title>
</head>
<body>
  <!-- 页面头部 -->
  <header class="site-header">
    <h1>网站标题</h1>
    <!-- 导航区域 -->
    <nav aria-label="主导航">
      <ul>
        <li><a href="/">首页</a></li>
        <li><a href="/about">关于</a></li>
        <li><a href="/contact">联系</a></li>
      </ul>
    </nav>
  </header>

  <!-- 主要内容区（每个页面只有一个） -->
  <main>
    <!-- 文章/独立内容块 -->
    <article>
      <header>
        <h2>文章标题</h2>
        <time datetime="2024-01-15">2024年1月15日</time>
      </header>

      <!-- 内容分区 -->
      <section>
        <h3>章节一</h3>
        <p>内容...</p>
      </section>

      <section>
        <h3>章节二</h3>
        <p>内容...</p>
      </section>

      <footer>
        <p>作者: 张三</p>
      </footer>
    </article>

    <!-- 侧边栏辅助内容 -->
    <aside>
      <h3>相关推荐</h3>
```

...

</aside>

</main>

<footer class="site-footer">

<p>© 2024 公司名称</p>

</footer>

</body>

</html>

□ 1.3 表单增强

```
<!-- 新输入类型 -->
<form action="/api/user" method="POST">
  <!-- 邮箱输入（自动验证格式） -->
  <label for="email">邮箱:</label>
  <input type="email" id="email" name="email" required>

  <!-- 数字输入（带步进器） -->
  <label for="age">年龄:</label>
  <input type="number" id="age" name="age" min="0" max="150" step="1">

  <!-- 日期选择器 -->
  <label for="birthday">生日:</label>
  <input type="date" id="birthday" name="birthday">

  <!-- 颜色选择器 -->
  <label for="color">主题色:</label>
  <input type="color" id="color" name="color" value="#ffffff">

  <!-- 搜索框（可清除按钮） -->
  <label for="search">搜索:</label>
  <input type="search" id="search" name="search" placeholder="输入
关键词...">

  <!-- URL输入 -->
  <label for="website">个人网站:</label>
  <input type="url" id="website" name="website" placeholder="https://...">

  <!-- 范围滑块 -->
  <label for="volume">音量:</label>
  <input type="range" id="volume" name="volume" min="0" max="100" value="50">

  <button type="submit">提交</button>
</form>
```

□ 2. CSS现代布局技术（1.5小时）

□ 2.1 Flexbox 弹性盒子布局

核心概念：

- ☐ 容器 (Container) - 设置display: flex 的父元素
- ☐ 项目 (Items) - 容器的直接子元素
- ☐ 主轴 (Main Axis) - 默认水平方向
- ☐ 交叉轴 (Cross Axis) - 与主轴垂直的方向

容器属性:

```
.container {
    display: flex;

    /* 主轴方向 */
    flex-direction: row;          /* 行排列 (默认) */
    /* flex-direction: row-reverse; */ /* 反向行 */
    /* flex-direction: column; */      /* 列 */
    /* flex-direction: column-reverse; */ /* 反向列 */

    /* 主轴对齐 */
    justify-content: flex-start; /* 起点 (默认) */
    /* justify-content: flex-end; */ /* 终点 */
    /* justify-content: center; */ /* 居中 */
    /* justify-content: space-between; */ /* 两端对齐 */
    /* justify-content: space-around; */ /* 环绕分布 */
    /* justify-content: space-evenly; */ /* 均匀分布 */

    /* 交叉轴对齐 */
    align-items: stretch;        /* 拉伸填满 (默认) */
    /* align-items: flex-start; */ /* 起点 */
    /* align-items: flex-end; */ /* 终点 */
    /* align-items: center; */ /* 居中 */
    /* align-items: baseline; */ /* 基线对齐 */

    /* 换行 */
    flex-wrap: nowrap;           /* 不换行 (默认) */
    /* flex-wrap: wrap; */       /* 换行 */
    /* flex-wrap: wrap-reverse; */ /* 反向换行 */

    /* 简写: 方向 + 换行 */
    /* flex-flow: row wrap; */
}
```

项目属性:

```

.item {
    /* 放大比例 */
    flex-grow: 0;                /* 不放大（默认） */
    /* flex-grow: 1; */         /* 等分剩余空间 */

    /* 缩小比例 */
    flex-shrink: 1;              /* 允许缩小（默认） */
    /* flex-shrink: 0; */       /* 不缩小 */

    /* 初始大小 */
    flex-basis: auto;            /* 内容大小（默认） */
    /* flex-basis: 200px; */     /* 固定宽度 */

    /* 简写：放大 + 缩小 + 初始 */
    /* flex: 1 1 auto; */

    /* flex: 0 0 200px; */       /* 固定200px不伸缩 */

    /* 单独设置交叉轴对齐 */
    align-self: center;          /* 覆盖容器的align-items */

    /* 排列顺序 */
    order: 0;                    /* 默认顺序 */
    /* order: -1; */             /* 排到前面 */
}

```

实战案例 - 导航栏：


```
<nav class="navbar">
  <div class="logo">ðŸ‡ª AI Agent</div>
  <ul class="nav-links">
    <li><a href="/">首页</a></li>
    <li><a href="/docs">文档</a></li>
    <li><a href="/pricing">价格</a></li>
  </ul>
  <button class="cta-button">开始使用</button>
</nav>
```

```
<style>
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 1rem 2rem;
  background: #ffffff;
  box-shadow: 0 2px 4px rgba(0,0,0,0.1);
}
```

*

```
.logo {
  font-size: 1.5rem;
  font-weight: bold;
}
```

*

```
.nav-links {
  display: flex;
  gap: 2rem;
  list-style: none;
  margin: 0;
  padding: 0;
}
```

*

```
.nav-links a {
  text-decoration: none;
  color: #333;
  transition: color 0.3s;
}
```

*

```
.nav-links a:hover {
  color: #007bff;
}
```

*

```
.cta-button {
  padding: 0.5rem 1.5rem;
  background: #007bff;
  color: white;
  border: none;
  border-radius: 4px;
}
```

```
cursor: pointer;
```

transition: background 0.3s;


```
.cta-button:hover {
```

background: #0056b3;

</style>

□ 2.2 CSS Grid 网格布局

二维布局系统 - 同时控制行和列:

```
.container {
    display: grid;

    /* 定义列宽 */
    grid-template-columns: 1fr 2fr 1fr;          /* 三列, 比例1:2:1 */
    /* grid-template-columns: repeat(3, 1fr); */ /* 三等分 */
    /* grid-template-columns: 200px 1fr 300px; */ /* 固定+自适应+固定 */

    /* 定义行高 */
    grid-template-rows: auto 1fr auto;          /* 三行 */

    /* 间距 */
    gap: 20px;                                  /* 行列统一间距 */
    /* row-gap: 10px; */                        /* 行间距 */
    /* column-gap: 15px; */                    /* 列间距 */

    /* 对齐方式 */
    place-items: center;                        /* 双轴居中 */
    /* justify-items: start; */                 /* 主轴对齐 */
    /* align-items: end; */                    /* 交叉轴对齐 */
}
```

子项定位:

```
.item {
    /* 指定位置 (网格线编号从1开始) */
    grid-column: 1 / 3;                        /* 从第1条线到第3条线 (跨2列) */
    grid-row: 2 / 4;                          /* 从第2条线到第4条线 (跨2行) */

    /* 简写 */
    /* grid-area: 2 / 1 / 4 / 3; */ /* row-start / col-start / row-end / col-end */

    /* 命名区域 (配合grid-template-areas使用) */
    /* grid-area: header; */
}
```

命名区域布局:

```

.page-layout {
  display: grid;
  grid-template-areas:
    "header header header"
    "sidebar main aside"
    "footer footer footer";
  grid-template-columns: 200px 1fr 200px;
  grid-template-rows: auto 1fr auto;
  min-height: 100vh;
  gap: 1rem;
}

.header { grid-area: header; *
.sidebar { grid-area: sidebar; *
.main { grid-area: main; *
.aside { grid-area: aside; *
.footer { grid-area: footer; *

/* 响应式调整 */
@media (max-width: 768px) {
  .page-layout {
    grid-template-areas:
      "header"
      "main"
      "sidebar"
      "aside"
      "footer";
    grid-template-columns: 1fr;
  }
}

```

□ 2.3 响应式设计

媒体查询:

```
/* 移动优先策略 */

/* 基础样式（移动端） */
.container {
    width: 100%;
    padding: 1rem;
}

/* 平板及以上 */
@media (min-width: 768px) {
    .container {
        max-width: 720px;
        margin: 0 auto;
        padding: 2rem;
    }
}

/* 桌面及以上 */
@media (min-width: 1024px) {
    .container {
        max-width: 960px;
    }
}

/* 大屏桌面 */
@media (min-width: 1280px) {
    .container {
        max-width: 1200px;
    }
}
```

现代响应式单位：

```
.element {  
    /* 视口相对单位 */  
    width: 50vw;           /* 视口宽度的50% */  
    height: 100vh;         /* 视口高度的100% */  
    font-size: clamp(1rem, 2vw, 1.5rem); /* 最小1rem, 首选2vw, 最大1.5rem */  
}  
  
/* 容器查询（新特性） */  
container-type: inline-size;  
*  
  
@container (min-width: 400px) {  
    .element {  
        font-size: 1.25rem;  
    }  
    *  
}
```

□ 3. JavaScript ES6+ 核心特性（1.5小时）

□ 3.1 变量声明与作用域

```

// var vs let vs const

// ❑ var - 函数作用域，存在变量提升
function example() {
    console.log(x); // undefined (不会报错)
    var x = 10;
}

// ❑ let - 块作用域，无变量提升（暂时性死区）
function modernExample() {
    // console.log(y); // ReferenceError
    let y = 20;
    if (true) {
        let z = 30; // z只在if块内可见
    }
    // console.log(z); // ReferenceError
}

// ❑ const - 块作用域，必须初始化，不可重新赋值
const API_URL = 'https://api.example.com';
// API_URL = 'new url'; // TypeError

// 对象和数组用const声明后，其内容仍可修改
const config = {
    debug: true,
    version: '1.0'
};

config.debug = false; // ❑合法
config.newProp = 'test'; // ❑合法
// config = {*; // ❑非法

```

❑ 3.2 箭头函数

```

// 传统函数
function add(a, b) {
    return a + b;
}

// 箭头函数
const add = (a, b) => a + b;

// 多行箭头函数
const multiply = (a, b) => {
    const result = a * b;
    return result;
};

// 单参数可省略括号
const double = x => x * 2;

// 无参数
const getRandom = () => Math.random();

// 返回对象（需加括号）
const createUser = (name, age) => ({
    name,
    age,
    createdAt: new Date()
});

// □注意：箭头函数没有自己的this
const user = {
    name: 'Alice',
    greet: function() {
        // 传统函数：this指向user对象
        console.log(`Hello, I'm ${this.name*}`);

        // 箭头函数：this继承自外层作用域
        setTimeout(() => {
            console.log(`Delayed: I'm ${this.name*}`); // this仍然是user
        }, 1000);
    }
};

```

□3.3 解构赋值

```

// 数组解构
const numbers = [1, 2, 3, 4, 5];
const [first, second, ...rest] = numbers;
console.log(first, second, rest); // 1 2 [3, 4, 5]

// 交换变量
let a = 1, b = 2;
[a, b] = [b, a]; // a=2, b=1

// 对象解构
const user = {
  name: 'Bob',
  age: 25,
  email: 'bob@example.com',
  address: {
    city: 'Beijing',
    country: 'China'
  }
};

const { name, age, email } = user;
console.log(name, age, email);

// 重命名
const { name: userName, age: userAge } = user;

// 默认值
const { gender = 'unknown' } = user;

// 嵌套解构
const {
  address: { city, country },
  ...otherInfo
} = user;

// 函数参数解构
function processUser({ name, age, role = 'user' }) {
  return `${name} is ${age} years old, role: ${role}`;
}

processUser(user); // "Bob is 25 years old, role: user"

```

3.4 模板字符串


```

const name = 'Charlie';
const items = ['Apple', 'Banana', 'Orange'];

// 多行字符串
const html = `
  <div class="card">
    <h2>${name}'s Shopping List</h2>
    <ul>
      ${items.map(item => `<li>${item}</li>`).join('')}
    </ul>
  </div>
`;

// 表达式插值
const price = 99.9;
const discounted = price * 0.8;
console.log(`原价: *${price}, 折后价: *${discounted.toFixed(2)}*`);

// 标签模板（高级用法）
function highlight(strings, ...values) {
  return strings.reduce((result, str, i) => {
    const value = values[i] ? `<strong>${values[i]}</strong>` : '';
    return result + str + value;
  }, '');
}

const keyword = 'AI';
const sentence = highlight`${keyword}* Agent是未来的趋势`;
// "<strong>AI</strong> Agent是未来的趋势"

```

3.5 Promise 与 async/await

```

// Promise基础
function fetchData(url) {
    return new Promise((resolve, reject) => {
        fetch(url)
            .then(response => response.json())
            .then(data => resolve(data))
            .catch(error => reject(error));
    });
}

// 使用Promise
fetchData('https://api.example.com/users')
    .then(users => console.log(users))
    .catch(error => console.error('Error:', error));

// async/await语法（更易读）
async function getUsers() {
    try {
        const users = await fetchData('https://api.example.com/users');
        console.log(users);

        // 并发请求
        const [posts, comments] = await Promise.all([
            fetchData('https://api.example.com/posts'),
            fetchData('https://api.example.com/comments')
        ]);

        return { users, posts, comments };
    } catch (error) {
        console.error('Failed to load data:', error);
        throw error;
    }
}

// 错误处理最佳实践
async function safeFetch(url, retries = 3) {
    for (let i = 0; i < retries; i++) {
        try {
            const response = await fetch(url);
            if (!response.ok) {
                throw new Error(`HTTP ${response.status}`);
            }
            return await response.json();
        } catch (error) {
            if (i === retries - 1) throw error;
            console.log(`Retry ${i + 1}/${retries}`);
            await new Promise(r => setTimeout(r, 1000 * (i + 1)));
        }
    }
}

// 指数退避

```


3.6 数组高阶方法

```
const users = [
  { id: 1, name: 'Alice', age: 28, active: true },
  { id: 2, name: 'Bob', age: 32, active: false },
  { id: 3, name: 'Carol', age: 24, active: true },
];

// map - 转换数组
const names = users.map(user => user.name);
// ['Alice', 'Bob', 'Carol']

// filter - 过滤数组
const activeUsers = users.filter(user => user.active);
// [{id:1,...}, {id:3,...}]

// find - 查找单个元素
const carol = users.find(user => user.name === 'Carol');

// reduce - 累计计算
const totalAge = users.reduce((sum, user) => sum + user.age, 0);
// 84

const ageByName = users.reduce((acc, user) => {
  acc[user.name] = user.age;
  return acc;
}, {});
// { Alice: 28, Bob: 32, Carol: 24 }

// some / every - 条件判断
const hasActiveUser = users.some(user => user.active); // true
const allActive = users.every(user => user.active);    // false

// includes - 存在性检查
const fruits = ['apple', 'banana', 'orange'];
const hasBanana = fruits.includes('banana'); // true

// 链式调用（组合多个方法）
const result = users
  .filter(user => user.active && user.age > 25)
  .map(user => ({
    ...user,
    category: user.age >= 30 ? 'senior' : 'junior'
  }))
  .sort((a, b) => a.age - b.age);
// [{id:1, name:'Alice', age:28, ..., category:'junior'}]
```

□3.7 对象与数组展开运算符

```
// 数组展开
const arr1 = [1, 2, 3];
const arr2 = [4, 5, 6];
const combined = [...arr1, ...arr2]; // [1,2,3,4,5,6]

// 复制数组（浅拷贝）
const copy = [...arr1];

// 函数参数展开
function sum(...numbers) {
  return numbers.reduce((total, n) => total + n, 0);
}
*
sum(1, 2, 3, 4, 5); // 15

// 对象展开
const defaults = {
  theme: 'light',
  language: 'zh',
  notifications: true
};

const userSettings = {
  theme: 'dark',
  fontSize: 16
};

const settings = { ...defaults, ...userSettings };
// { theme: 'dark', language: 'zh', notifications: true, fontSize: 16 }
// 后面的属性会覆盖前面的

// React/Vue中常用（不可变更新）
const state = {
  users: [],
  loading: false,
  error: null
};

// 更新loading状态
const newState = { ...state, loading: true };

// 更新嵌套对象
const updatedState = {
  ...state,
  users: [...state.users, newUser]
};
```

□4. Vue3 Composition API 深度学习 (2小时)

□4.1 从Options API到Composition API

Options API的问题:

```
<!-- Options API: 逻辑分散 -->
<script>
export default {
  data() {
    return {
      user: null,
      loading: false,
      error: null
    }
  },
  computed: {
    isLoggedIn() {
      return !!this.user
    }
  },
  methods: {
    async fetchUser() {
      this.loading = true
      try {
        this.user = await api.getUser()
      } catch (e) {
        this.error = e.message
      } finally {
        this.loading = false
      }
    }
  },
  mounted() {
    this.fetchUser()
  }
}
</script>
```

Composition API的优势:

```
<!-- Composition API: 逻辑聚合 -->
<script setup>
import { ref, computed, onMounted } from 'vue'

// 用户相关的所有逻辑放在一起
const user = ref(null)
const loading = ref(false)
const error = ref(null)

const isLoggedIn = computed(() => !!user.value)

async function fetchUser() {
  loading.value = true
  try {
    user.value = await api.getUser()
  } catch (e) {
    error.value = e.message
  } finally {
    loading.value = false
  }
}

onMounted(() => {
  fetchUser()
})
</script>
```

□4.2 核心API详解

ref 和 reactive:


```

<script setup>
import { ref, reactive, toRefs * from 'vue'

// ref - 基本类型或需要替换的对象
const count = ref(0)
const message = ref('Hello')
const user = ref({ name: 'Alice', age: 28 *})

// 访问值需要.value
console.log(count.value) // 0
count.value++

// reactive - 对象/数组（不需要.value）
const state = reactive({
  list: [],
  filters: {
    keyword: '',
    status: 'all'
  },
  pagination: {
    page: 1,
    pageSize: 10
  }
})

// 直接访问
state.list.push(item)
state.filters.keyword = 'search'

// ❑解构reactive会失去响应性
const { keyword, status * = state.filters // ❑失去响应性

// 解决方案1: toRefs
const { keyword, status * = toRefs(state.filters) // ❑保持响应性

// 解决方案2: 继续使用reactive对象
// 推荐在template中直接使用state.xxx
</script>

<template>
  <div>
    <p>Count: {{ count **</p>
    <button @click="count++">Increment</button>

    <p>User: {{ user.name **</p>
    <input v-model="state.filters.keyword" />
  </div>
</template>

```

computed 计算属性:

```

<script setup>
import { ref, computed } from 'vue'

const products = ref([
  { id: 1, name: 'Laptop', price: 9999, stock: 10 },
  { id: 2, name: 'Phone', price: 5999, stock: 0 },
  { id: 3, name: 'Tablet', price: 3999, stock: 5 }
])

const filterText = ref('')
const showOutOfStock = ref(false)

// 过滤后的商品列表
const filteredProducts = computed(() => {
  let result = products.value

  if (filterText.value) {
    result = result.filter(p =>
      p.name.toLowerCase().includes(filterText.value.toLowerCase())
    )
  }

  if (!showOutOfStock.value) {
    result = result.filter(p => p.stock > 0)
  }

  return result
})

// 统计信息
const stats = computed(() => {
  const total = filteredProducts.value.length
  const inStock = filteredProducts.value.filter(p => p.stock > 0).length
  const totalPrice = filteredProducts.value.reduce((sum, p) => sum + p.price, 0)

  return { total, inStock, avgPrice: total ? Math.round(totalPrice / total) : 0 }
})
</script>

<template>
<div>
  <input v-model="filterText" placeholder="搜索商品..." />
  <label>
    <input type="checkbox" v-model="showOutOfStock" />
    显示缺货商品
  </label>

  <p>共 {{ stats.total }} 件商品, {{ stats.inStock }} 件有货, 均价 *{{ stats.avgPrice }}</p>

```



```
<li v-for="product in filteredProducts" :key="product.id">
```

{{ product.name ** - *{{ product.price ** (库存: {{ product.

stock **)

</div>

</template>

watch 和 watchEffect 监听器:

```

<script setup>
import { ref, watch, watchEffect * from 'vue'

const question = ref('')
const answer = ref('请输入问题...')
const loading = ref(false)

// watch: 明确指定监听源
watch(question, async (newQuestion, oldQuestion) => {
  if (!newQuestion.includes('?')) {
    answer.value = '问题需要包含问号❏'
    return
  }

  loading.value = true
  answer.value = '思考中...'

  try {
    // 模拟API调用
    await new Promise(resolve => setTimeout(resolve, 1000))
    answer.value = `关于"${newQuestion}"的答案是...`
  } finally {
    loading.value = false
  }
})

// watchEffect: 自动追踪依赖
const searchText = ref('')
const results = ref([])

watchEffect(async () => {
  // 自动追踪searchText的变化
  if (searchText.value.length < 2) {
    results.value = []
    return
  }

  // 执行搜索
  results.value = await searchAPI(searchText.value)
})

// watch选项配置
const deepObject = ref({
  nested: {
    value: 0
  }
})

watch(

```

```
() => deepObject.value.nested.value, // getter函数
```

```
(newVal, oldVal) => {
```



```
console.log('Nested value changed:', newVal)
```


{

```
immediate: true,    // 立即执行一次
```

```
deep: true,          // 深度监听
```

```
flush: 'post'      // DOM更新后执行（默认pre）
```


</script>

生命周期钩子:

```
<script setup>
import {
  onMounted,
  onUnmounted,
  onUpdated,
  onBeforeMount,
  onBeforeUnmount,
  nextTick
} from 'vue'

onBeforeMount(() => {
  console.log('组件即将挂载')
})

onMounted(() => {
  console.log('组件已挂载, DOM可用')

  // 获取DOM元素
  nextTick(() => {
    // 在DOM更新完成后执行
    const el = document.querySelector('.my-element')
    console.log(el.offsetHeight)
  })
})

onUpdated(() => {
  console.log('组件数据更新, DOM重新渲染')
})

onBeforeUnmount(() => {
  console.log('组件即将卸载')
})

onUnmounted(() => {
  console.log('组件已卸载, 清理定时器/事件监听')
  clearInterval(timer)
  window.removeEventListener('resize', handleResize)
})
</script>
```

□4.3 组合式函数 (Composables)

提取复用逻辑:

```
// composables/useCounter.js
import { ref, computed * from 'vue'

export function useCounter(initialValue = 0) {
  const count = ref(initialValue)

  const increment = () => count.value++
  const decrement = () => count.value--
  const reset = () => count.value = initialValue

  const doubled = computed(() => count.value * 2)

  return {
    count,
    increment,
    decrement,
    reset,
    doubled
  }
}
*
```

```
<!-- 使用composable -->
<script setup>
import { useCounter * from './composables/useCounter'

const { count, increment, decrement, reset, doubled * = useCounter(10)
</script>

<template>
  <div>
    <p>Count: {{ count }} (Doubled: {{ doubled }})</p>
    <button @click="increment">+</button>
    <button @click="decrement">-</button>
    <button @click="reset">Reset</button>
  </div>
</template>
```

实际案例 - 数据获取Composable:

```

// composables/useAsync.js
import { ref, isRef, unref, watchEffect } from 'vue'

export function useAsync(asyncFn, options = { * }) {
  const {
    initialData = null,
    immediate = true,
    onSuccess,
    onError
  } = options

  const data = ref(initialData)
  const error = ref(null)
  const loading = ref(false)

  async function execute(...args) {
    loading.value = true
    error.value = null

    try {
      const result = await asyncFn(...args)
      data.value = result
      onSuccess?.(result)
      return result
    } catch (e) {
      error.value = e.message
      onError?.(e)
      throw e
    } finally {
      loading.value = false
    }
  }

  if (immediate) {
    watchEffect(() => execute(unref(isRef(asyncFn) ? asyncFn.value : asyncFn)))
  }

  return {
    data,
    error,
    loading,
    execute
  }
}

```

```

<script setup>
import { useAsync * from './composables/useAsync'

// 获取用户列表
const { data: users, loading, error, execute: fetchUsers * = useAsync(
  () => api.getUsers(),
  {
    initialData: [],
    onSuccess: (data) => console.log('Loaded', data.length, 'users'),
    onError: (e) => console.error('Failed:', e)
  }
  *
)

// 手动刷新
function refresh() {
  fetchUsers()
  *
}
</script>

<template>
  <div v-if="loading">加载中...</div>
  <div v-else-if="error">错误: {{ error }}</div>
  <ul v-else>
    <li v-for="user in users" :key="user.id">{{ user.name }}</li>
  </ul>

  <button @click="refresh" :disabled="loading">刷新</button>
</template>

```

5. 组件通信与状态管理 (1.5小时)

5.1 Props 和 Emits

父传子 - Props:

```
<!-- Parent.vue -->
<script setup>
import ChildComponent from './ChildComponent.vue'

const userData = {
  name: 'Alice',
  avatar: '/avatar.jpg',
  roles: ['admin', 'editor']
}

*

function handleUpdate(newName) {
  userData.name = newName
}

*
</script>

<template>
  <ChildComponent
    :user="userData"
    :is-admin="true"
    title="用户信息"
    @update="handleUpdate"
  />
</template>
```

```

<!-- ChildComponent.vue -->
<script setup>
// 定义props及其类型和默认值
const props = defineProps({
  user: {
    type: Object,
    required: true
  },
  isAdmin: {
    type: Boolean,
    default: false
  },
  title: {
    type: String,
    default: '默认标题'
  }
})

// 定义emits事件
const emit = defineEmits(['update', 'delete'])

function updateName() {
  emit('update', 'New Name')
}

</script>

<template>
  <div class="child-component">
    <h3>{{ title }}</h3>
    
    <p>{{ user.name }}</p>
    <span v-for="role in user.roles" :key="role">{{ role }}</span>

    <button @click="updateName">修改名称</button>
  </div>
</template>

```

TypeScript支持:

```

// 使用TypeScript接口定义props
interface User {
  name: string
  avatar: string
  roles: string[]
}

interface Props {
  user: User
  isAdmin?: boolean
  title?: string
}

// 使用泛型定义
const props = withDefaults(defineProps<Props>(), {
  isAdmin: false,
  title: '默认标题'
})

// 类型安全的emit
const emit = defineEmits<{
  update: [name: string]
  delete: [id: number]
}>()

emit.update('New Name') // TypeScript会检查参数类型

```

5.2 依赖注入 (Provide/Inject)

跨层级通信：


```

<!-- App.vue (根组件) -->
<script setup>
import { provide, ref, readonly * from 'vue'
import ParentComponent from './ParentComponent.vue'

// 提供全局状态
const theme = ref('light')
const currentUser = ref(null)

// 提供方法
function toggleTheme() {
  theme.value = theme.value === 'light' ? 'dark' : 'light'
  *

function login(user) {
  currentUser.value = user
  *

// 使用readonly防止子组件修改
provide('theme', readonly(theme))
provide('toggleTheme', toggleTheme)
provide('currentUser', readonly(currentUser))
provide('login', login)
</script>

```

```

<!-- DeepChild.vue (深层子组件) -->
<script setup>
import { inject * from 'vue'

// 注入父级提供的值和方法
const theme = inject('theme')
const toggleTheme = inject('toggleTheme')
const currentUser = inject('currentUser')

// 提供默认值
const appVersion = inject('version', '1.0.0')
</script>

<template>
  <div :class="`theme-${theme}`">
    <p>当前主题: {{ theme }}</p>
    <button @click="toggleTheme">切换主题</p>
    <p v-if="currentUser">欢迎, {{ currentUser.name }}</p>
  </div>
</template>

```

□ 5.3 Pinia 状态管理

安装与配置:

```
pnpm add pinia
```

```
// main.js
import { createApp } from 'vue'
import { createPinia } from 'pinia'
import App from './App.vue'

const app = createApp(App)
app.use(createPinia())
app.mount('#app')
```

定义Store:

```

// stores/userStore.js
import { defineStore * from 'pinia'
import { ref, computed * from 'vue'
import { authApi * from '@api/auth'

export const useUserStore = defineStore('user', () => {
  // State (相当于data)
  const user = ref(null)
  const token = ref(localStorage.getItem('token') || '')
  const loading = ref(false)
  const error = ref(null)

  // Getters (相当于computed)
  const isLoggedIn = computed(() => !!token.value && !!user.value)
  const userName = computed(() => user.value?.name || 'Guest')
  const userRoles = computed(() => user.value?.roles || [])

  // Actions (相当于methods)
  async function login(credentials) {
    loading.value = true
    error.value = null

    try {
      const response = await authApi.login(credentials)
      token.value = response.token
      user.value = response.user

      localStorage.setItem('token', response.token)

      return response
    } catch (e) {
      error.value = e.message
      throw e
    } finally {
      loading.value = false
    }
  }

  async function fetchUser() {
    if (!token.value) return

    try {
      user.value = await authApi.getCurrentUser()
    } catch (e) {
      logout()
    }
  }

  function logout() {

```

```
user.value = null
```

```
token.value = ''
```

```
localStorage.removeItem('token')
```


// 返回需要在组件中使用的内容

```
return {
```

```
// State
```

user,

token,

loading,

error,

```
// Getters
```


isLoggedIn,

userName,

userRoles,

// Actions

login,

fetchUser,

[logout](#)

在组件中使用：

```
<script setup>
import { useUserStore * from '@/stores/userStore'
import { storeToRefs * from 'pinia'

const userStore = useUserStore()

// 使用storeToRefs保持响应性（解构state和getters）
const { user, isLoggedIn, userName, loading * = storeToRefs(userStore)

// actions可以直接解构
const { login, logout, fetchUser * = userStore

// 登录表单
async function handleLogin(formData) {
  try {
    await login(formData)
    // 登录成功后的操作
  * catch (e) {
    console.error('Login failed:', e)
  *
  *
}
</script>

<template>
  <div v-if="loading">登录中...</div>
  <div v-else-if="isLoggedIn">
    <p>欢迎回来, {{ userName }}</p>
    <button @click="logout">退出登录</button>
  </div>
  <div v-else>
    <!-- 登录表单 -->
    <form @submit.prevent="handleLogin">
      <input v-model="username" placeholder="用户名" />
      <input v-model="password" type="password" placeholder="密码" />
    >
    <button type="submit">登录</button>
  </form>
  </div>
</template>
```

多Store协作：

```
// stores/appStore.js
export const useAppStore = defineStore('app', () => {
  const sidebarCollapsed = ref(false)
  const theme = ref(localStorage.getItem('theme') || 'light')

  function toggleSidebar() {
    sidebarCollapsed.value = !sidebarCollapsed.value
  }

  function setTheme(newTheme) {
    theme.value = newTheme
    localStorage.setItem('theme', newTheme)
  }

  return {
    sidebarCollapsed,
    theme,
    toggleSidebar,
    setTheme
  }
})
```

```
<script setup>
// 同时使用多个store
import { useUserStore * from '@/stores/userStore'
import { useAppStore * from '@/stores/appStore'

const userStore = useUserStore()
const appStore = useAppStore()

// 可以在不同store间交互
function handleUserLogin() {
  // userStore登录成功后
  appStore.setTheme(userStore.user.preferences.theme || 'light')
}
</script>
```

6. Vue Router 路由管理 (1小时)

6.1 基础路由配置

```

// router/index.js
import { createRouter, createWebHistory } from 'vue-router'
import HomeView from '@views/HomeView.vue'
import AboutView from '@views/AboutView.vue'
import LoginView from '@views/LoginView.vue'
import DashboardView from '@views/DashboardView.vue'
import NotFoundView from '@views/NotFoundView.vue'

const routes = [
  {
    path: '/',
    name: 'home',
    component: HomeView,
    meta: { title: '首页', requiresAuth: false }
  },
  {
    path: '/about',
    name: 'about',
    component: AboutView,
    meta: { title: '关于我们' }
  },
  {
    path: '/login',
    name: 'login',
    component: LoginView,
    meta: { guestOnly: true } // 仅未登录可访问
  },
  {
    path: '/dashboard',
    name: 'dashboard',
    component: DashboardView,
    meta: { requiresAuth: true, title: '控制台' }
  },
  // 动态路由
  {
    path: '/user/:id',
    name: 'user-profile',
    component: () => import('@views/UserProfile.vue'), // 懒加载
    props: true // 将route.params作为props传递给组件
  },
  // 404页面（必须放在最后）
  {
    path: '/*',
    name: 'not-found',
    component: NotFoundView
  }
]

const router = createRouter({

```

```
history: createWebHistory(import.meta.env.BASE_URL),
```

routes,

```
scrollBehavior(to, from, savedPosition) {
```

// 路由切换时滚动行为


```
if (savedPosition) {
```

```
return savedPosition
```

```
* else {
```

```
return { top: 0 *
```


// 全局前置守卫

```
router.beforeEach((to, from, next) => {
```

// 设置页面标题

```
document.title = to.meta.title || 'AI Agent App'
```


// 认证检查

```
const userStore = useUserStore()
```



```
if (to.meta.requiresAuth && !userStore.isLoggedIn) {
```

// 需要登录但未登录，重定向到登录页

```
next({
```

name: 'login',

```
query: { redirect: to.fullPath * // 保存原始路径
```



```
* else if (to.meta.guestOnly && userStore.isLoggedIn) {
```

// 仅游客页面但已登录，重定向到首页


```
next({ name: 'home' *})
```

```
* else {
```

next()

export router

□6.2 程式导航


```
<script setup>
import { useRouter, useRoute * from 'vue-router'

const router = useRouter()
const route = useRoute()

// 基础导航
function navigateHome() {
  router.push('/')

  // 或
  router.push({ name: 'home' *})
*

// 带参数导航
function viewUserProfile(userId) {
  router.push({
    name: 'user-profile',
    params: { id: userId *}
  *})
  // URL: /user/123
*

// 带查询参数
function search(keyword) {
  router.push({
    path: '/search',
    query: { q: keyword, page: 1 *}
  *})
  // URL: /search?q=keyword&page=1
*

// 替换历史记录（不产生新的历史记录）
function replaceCurrentRoute() {
  router.replace({ name: 'dashboard' *})
*

// 前进/后退
function goBack() {
  router.go(-1) // 或 router.back()
*

function goForward() {
  router.go(1) // 或 router.forward()
*

// 导航守卫（组件内）
onBeforeRouteLeave((to, from) => {
  // 离开确认
  if (hasUnsavedChanges) {
```

```
const answer = window.confirm('确定离开？未保存的数据将会丢失。')
```

```
if (!answer) return false
```



```
onBeforeRouteUpdate(async (to, from) => {
```

// 同一组件，参数变化时重新获取数据


```
if (to.params.id !== from.params.id) {
```

```
userData.value = await fetchUser(to.params.id)
```


</script>

6.3 嵌套路由和布局

```
// router/index.js
const routes = [
  {
    path: '/admin',
    component: AdminLayout, // 布局组件
    meta: { requiresAuth: true, requiresAdmin: true },
    children: [
      {
        path: '', // /admin
        name: 'admin-dashboard',
        component: AdminDashboard
      },
      {
        path: 'users', // /admin/users
        name: 'admin-users',
        component: AdminUsers
      },
      {
        path: 'settings', // /admin/settings
        name: 'admin-settings',
        component: AdminSettings
      }
    ]
  }
]
```

```
<!-- AdminLayout.vue -->
<template>
  <div class="admin-layout">
    <aside class="sidebar">
      <nav>
        <router-link to="/admin">仪表盘</router-link>
        <router-link to="/admin/users">用户管理</router-link>
        <router-link to="/admin/settings">系统设置</router-link>
      </nav>
    </aside>

    <main class="content">
      <!-- 子路由出口 -->
      <router-view />
    </main>
  </div>
</template>
```

*> 实践任务清单

☐任务1：创建待办事项应用（2小时）

功能需求：

- ☐ ✓ 添加待办事项（输入框 + 按钮）
- ☐ ✓ 标记完成/未完成（点击切换）
- ☐ ✓ 删除待办事项（删除按钮）
- ☐ ✓ 筛选显示（全部/已完成/未完成）
- ☐ ✓ 本地持久化（localStorage）
- ☐ ✓ 响应式设计（移动端适配）

代码骨架：

```

<!-- TodoApp.vue -->
<script setup>
import { ref, computed, watch * from 'vue'

// 状态
const todos = ref(JSON.parse(localStorage.getItem('todos') || '[]'))
const newTodo = ref('')
const filter = ref('all') // all | completed | active

// 自动保存到localStorage
watch(todos, (newVal) => {
  localStorage.setItem('todos', JSON.stringify(newVal))
  *, { deep: true *})

// 计算属性
const filteredTodos = computed(() => {
  switch (filter.value) {
    case 'completed':
      return todos.value.filter(todo => todo.completed)
    case 'active':
      return todos.value.filter(todo => !todo.completed)
    default:
      return todos.value
  }
  *
  *)

const stats = computed(() => {
  const total = todos.value.length
  const completed = todos.value.filter(t => t.completed).length
  return { total, completed, remaining: total - completed *
  *)

// 方法
function addTodo() {
  const text = newTodo.value.trim()
  if (!text) return

  todos.value.push({
    id: Date.now(),
    text,
    completed: false,
    createdAt: new Date().toISOString()
  })

  newTodo.value = ''
  *

function toggleTodo(id) {
  const todo = todos.value.find(t => t.id === id)

```



```
if (todo) todo.completed = !todo.completed
```



```
function deleteTodo(id) {
```

```
todos.value = todos.value.filter(t => t.id !== id)
```



```
function clearCompleted() {
```



```
todos.value = todos.value.filter(t => !t.completed)
```


</script>

<template>

```
<div class="todo-app">
```

<h1>□待办事项</h1>

<!-- 输入区域 -->

```
<form @submit.prevent="addTodo" class="input-group">
```

<input

v-model="newTodo"

type="text"

placeholder="添加新的待办事项..."

autofocus

/>

<button type="submit">添加</button>

</form>

<!-- 筛选器 -->

```
<div class="filters">
```

<button

```
v-for="f in ['all', 'active', 'completed']"
```

:key="f"


```
:class="{ active: filter === f *"
```

@click="filter = f"


```
{{ f === 'all' ? '全部' : f === 'active' ? '未完成' : '已完成' **
```

</button>

</div>

<p class="stats">

共 {{ stats.total ** }} 项, 已完成 {{ stats.completed ** }} 项,

剩余 {[stats.remaining ** 项

</p>

<!-- 待办列表 -->

```
<transition-group name="list" tag="ul" class="todo-list">
```

<li


```
v-for="todo in filteredTodos"
```

:key="todo.id"

```
:class="{ completed: todo.completed }"
```


<input

type="checkbox"

:checked="todo.completed"

```
@change="toggleTodo(todo.id)"
```


{{ todo.text }}

```
<button @click="deleteTodo(todo.id)" class="delete-btn">*</button>
```


</transition-group>

<!-- 清除已完成 -->

<button


```
v-if="stats.completed > 0"
```

@click="clearCompleted"

class="clear-completed"

清除已完成 ([[stats.completed **)

</button>

</div>

</template>

<style scoped>

```
.todo-app {
```

max-width: 600px;

```
margin: 2rem auto;
```

```
padding: 2rem;
```

font-family: system-ui, sans-serif;


```
.input-group {
```

```
display: flex;
```

gap: 0.5rem;

```
margin-bottom: 1rem;
```



```
.input-group input {
```



```
flex: 1;
```

padding: 0.75rem;

font-size: 1rem;

border: 2px solid #ddd;

`border-radius: 4px;`


```
.input-group button {
```



```
padding: 0.75rem 1.5rem;
```

background: #4CAF50;

color: white;

border: none;

```
border-radius: 4px;
```

cursor: pointer;


```
.filters {
```

```
display: flex;
```

gap: 0.5rem;

```
margin-bottom: 1rem;
```



```
.filters button {
```

```
padding: 0.5rem 1rem;
```


border: 2px solid #ddd;

```
background: white;
```

```
border-radius: 4px;
```

cursor: pointer;


```
.filters button.active {
```

background: #007bff;

color: white;

border-color: #007bff;


```
.todo-list {
```

list-style: none;

```
padding: 0;
```



```
.todo-list li {
```

```
display: flex;
```

```
align-items: center;
```

gap: 1rem;

```
padding: 1rem;
```

```
margin-bottom: 0.5rem;
```

background: #f9f9f9;


```
border-radius: 4px;
```

transition: all 0.3s;


```
.todo-list li.completed span {
```

text-decoration: line-through;

opacity: 0.6;


```
.delete-btn {
```

```
margin-left: auto;
```

background: #ff4444;

color: white;

border: none;

```
border-radius: 50%;
```

width: 24px;

height: 24px;

cursor: pointer;


```
.clear-completed {
```

```
margin-top: 1rem;
```

```
padding: 0.5rem 1rem;
```

background: #ff9800;

color: white;

border: none;

```
border-radius: 4px;
```

```
cursor: pointer;
```


/* 列表动画 */

.list-enter-active,


```
.list-leave-active {
```

transition: all 0.3s ease;

.list-enter-from,

```
.list-leave-to {
```

opacity: 0;

```
transform: translateX(-30px);
```


/* 响应式 */

```
@media (max-width: 480px) {
```

```
.todo-app {
```

padding: 1rem;


```
.filters {
```

```
flex-wrap: wrap;
```


</style>

检验标准：

- ☐ [] 能正常添加、删除、标记待办事项
- ☐ [] 筛选功能正确工作
- ☐ [] 刷新页面后数据保持（localStorage生效）
- ☐ [] 移动端显示正常（响应式布局）
- ☐ [] 有平滑的过渡动画效果

☐ 任务2：整合Vue Router和Pinia（1.5小时）

任务要求：

- ☐ 创建至少3个页面（首页、待办事项、关于页面）
- ☐ 配置路由和导航守卫
- ☐ 使用Pinia管理全局状态（如用户信息、主题）
- ☐ 实现导航栏组件（显示当前页面、链接跳转）

步骤提示：

- ☐ 安装依赖：

```
pnpm add vue-router pinia
```

- ☐ 创建路由配置（参考上文6.1节）
- ☐ 创建Store（参考上文5.3节）
- ☐ 创建布局组件和导航栏
- ☐ 实现页面间的数据共享

检验标准：

- ☐ [] 页面能正常跳转且URL变化正确
- ☐ [] 导航栏高亮当前页面
- ☐ [] Pinia状态在各页面间共享
- ☐ [] 浏览器前进/后退正常工作

** 推荐学习资源

☐ 官方文档

- ☐ MDN Web Docs - Web技术权威参考
- ☐ Vue3 官方文档 - Vue3完整指南
- ☐ Pinia 官方文档 - 状态管理
- ☐ Vue Router 官方文档 - 路由管理

☐ 推荐教程

- ☐ Vue Mastery (英文) - Vue官方推荐的视频课程
- ☐ Vue School (英文) - 系统化的Vue3课程
- ☐ 慕课网/Vue系列 (中文) - 国内优质Vue教程

☐ GitHub项目

- ☐ awesome-vue - Vue资源大全
- ☐ vue-vben-admin - 企业级后台管理系统模板

☐ 练习平台

- ☐ Vue.js Exercises - Vue练习题库
- ☐ CodePen - 在线前端代码编辑器

✓ 今日自测题

☐ 选择题 (每题10分, 共100分)

- ☐ 以下哪个不是HTML5语义化标签?
 - ☐ A.
 - ☐ B.
 - ☐ C.
 - ☐ D.
- ☐ Flexbox中justify-content 控制的是哪个方向的对齐?
 - ☐ A. 主轴方向
 - ☐ B. 交叉轴方向
 - ☐ C. 两个方向都控制
 - ☐ D. 取决于flex-direction
- ☐ Vue3 Composition API中, ref() 和reactive() 的主要区别?
 - ☐ A. 没有区别
 - ☐ B. ref用于基本类型, reactive用于对象
 - ☐ C. ref需要.value访问, reactive不需要
 - ☐ D. B和C都是
- ☐ 以下哪个ES6特性可以用来合并数组?
 - ☐ A. 解构赋值
 - ☐ B. 展开运算符(...)
 - ☐ C. Promise
 - ☐ D. 箭头函数

- ☐ Pinia中的storeToRefs() 的作用是什么?
 - ☐ A. 创建新的store
 - ☐ B. 保持解构出来的state/getters的响应性
 - ☐ C. 将store转换为只读
 - ☐ D. 重置store状态

- ☐ Vue Router中，路由懒加载的好处是什么?
 - ☐ A. 代码更简洁
 - ☐ B. 减少首屏加载时间
 - ☐ C. 提高运行性能
 - ☐ D. 以上都是

- ☐ CSS Grid和Flexbox的主要区别?
 - ☐ A. Grid是一维布局，Flexbox是二维布局
 - ☐ B. Grid是二维布局，Flexbox是一维布局
 - ☐ C. Grid只能用于表格
 - ☐ D. 没有本质区别

- ☐ async/await 相比 Promise.then() 的优势?
 - ☐ A. 性能更好
 - ☐ B. 代码更易读，像同步代码
 - ☐ C. 支持更多浏览器
 - ☐ D. 可以处理更多错误

- ☐ Vue3的 语法糖的作用?
 - ☐ A. 自动导入组件
 - ☐ B. 简化Composition API写法
 - ☐ C. 提供更好的TypeScript支持
 - ☐ D. 以上都是

- ☐ 响应式设计中，媒体查询的min-width: 768px 表示?
 - ☐ A. 最大宽度为768px
 - ☐ B. 最小宽度为768px
 - ☐ C. 正好等于768px
 - ☐ D. 高度为768px

☐答案与解析

- ☐ 答案: C - 是通用容器, 没有语义含义。
- ☐ 答案: A - justify-content 控制主轴方向的对齐, align-items 控制交叉轴。
- ☐ 答案: D - ref通常用于基本类型(也可用于对象), 访问需要.value; reactive用于对象/数组, 直接访问属性。
- ☐ 答案: B - [...arr1, ...arr2] 可以合并数组。
- ☐ 答案: B - 直接解构store会丢失响应性, storeToRefs可以将state和getters转换为响应式refs。
- ☐ 答案: B - 懒加载将代码分割成小块, 按需加载, 减少初始加载体积。
- ☐ 答案: B - Grid擅长二维布局(行和列), Flexbox擅长一维布局(行或列)。
- ☐ 答案: B - async/await让异步代码看起来像同步代码, 避免回调地狱, 更易读易维护。
- ☐ 答案: D - script setup自动注册组件、简化代码、提供更好的TS支持和编译优化。
- ☐ 答案: B - min-width: 768px 表示视口宽度 $\geq 768\text{px}$ 时应用样式(平板及以上设备)。

评分标准:

- ☐ 90-100分: ✓>> 优秀! 前端基础扎实!
- ☐ 70-89分: *☐良好! 建议复习错题知识点
- ☐ 60-69分: !! 及格! 需要加强练习
- ☐ 60分以下: *☐建议重新学习今日内容

** 今日总结

☐ 关键收获

- ☐ _____
- ☐ _____
- ☐ _____

☐ 遇到的挑战

- ☐ 问题: 解决:
- ☐ 问题: 解决:

☐ 明日预告

Day 3: 后端开发技能提升 将涵盖:

- ☐ Python高级特性和异步编程
- ☐ FastAPI框架深度实践
- ☐ RESTful API设计原则
- ☐ 数据验证与错误处理
- ☐ 数据库ORM集成

准备工作:

- ☐ [] 复习Python基础知识
- ☐ [] 预习FastAPI官方文档入门部分
- ☐ [] 安装PostgreSQL数据库（可选Docker方式）

** 实战项目：TodoMVC 待办事项应用

项目概览

项目名称: Todo App - Vue3待办事项管理应用 路径:02-前端技术复习与强化/todo-app/ 完成度: ✓
100% 文件数: 9个核心文件 技术栈: Vue3 + Vite + Composition API + Pinia + localStorage

核心特性

✓ 完整CRUD操作 - 添加/删除/标记完成待办事项 ✓ 筛选功能 - 全部/已完成/未完成三种视图 ✓
本地持久化 - localStorage自动保存, 刷新不丢失 ✓ 统计面板 - 实时显示总数/已完成/剩余数量
✓ 批量操作 - 一键清除所有已完成项 ✓ 过渡动画 - Vue3 平滑效果 ✓ 响应式设计 - 移动端适配, Flexbox布局

项目架构

```
todo-app/
+-- src/
|   +-- App.vue           # 主应用组件
|   +-- components/
|   |   +-- TodoItem.vue  # 单个待办项组件
|   +-- composables/
|   |   +-- useTodo.js    # Composable状态管理
|   +-- assets/
|   |   +-- main.css      # 全局样式
|   +-- main.js           # 应用入口
|
+-- index.html            # HTML入口
+-- package.json          # 依赖配置
+-- vite.config.js        # Vite构建配置
+-- README.md             # 项目说明
```

核心代码亮点

Composable模式 (useTodo.js)

```

// composables/useTodo.js
import { ref, computed, watch * from 'vue'

export function useTodo() {
  const todos = ref(JSON.parse(localStorage.getItem('todos') || '[]'))
  const filter = ref('all') // all | completed | active

  // 自动持久化
  watch(todos, (newVal) => {
    localStorage.setItem('todos', JSON.stringify(newVal))
  }, { deep: true })

  const filteredTodos = computed(() => {
    switch (filter.value) {
      case 'completed': return todos.value.filter(t => t.completed)
      case 'active': return todos.value.filter(t => !t.completed)
      default: return todos.value
    }
  })

  const stats = computed(() => ({
    total: todos.value.length,
    completed: todos.value.filter(t => t.completed).length,
    remaining: todos.value.filter(t => !t.completed).length
  }))

  function addTodo(text) { /* ... */ }
  function toggleTodo(id) { /* ... */ }
  function deleteTodo(id) { /* ... */ }

  return { todos, filter, filteredTodos, stats, addTodo, toggleTodo, deleteTodo }
}

```

□ 组件使用示例 (App.vue)

```
<script setup>
import { useTodo * from './composables/useTodo'

const { filteredTodos, stats, addTodo, toggleTodo, deleteTodo * = useTodo()
const newTodo = ref('')
</script>

<template>
  <div class="todo-app">
    <form @submit.prevent="addTodo(newTodo)">
      <input v-model="newTodo" placeholder="添加新的待办事项..." />
    </form>

    <div class="stats">
      共 {{ stats.total ** 项, 已完成 {{ stats.completed ** 项
    </div>

    <transition-group name="list" tag="ul">
      <li v-for="todo in filteredTodos" :key="todo.id">
        <input :checked="todo.completed" @change="toggleTodo(todo.id)" />
        <span>{{ todo.text **</span>
        <button @click="deleteTodo(todo.id)">*</button>
      </li>
    </transition-group>
  </div>
</template>
```

□快速启动

```
# 1. 进入项目目录
cd 02-前端技术复习与强化/todo-app

# 2. 安装依赖（首次运行）
npm install
# 或使用 pnpm install（推荐）

# 3. 启动开发服务器
npm run dev
# 或 pnpm dev

# 4. 访问应用
# http://localhost:5173（Vite默认端口）

# 5. 构建生产版本
npm run build
# 输出目录：dist/

# 6. 预览生产构建
npm run preview
```

□功能演示

- ☐ 添加待办：在输入框输入内容，按回车或点击添加按钮
- ☐ 标记完成：点击复选框切换完成状态，文字会显示删除线
- ☐ 删除待办：点击右侧 × 按钮删除单项
- ☐ 筛选视图：点击顶部筛选按钮（全部/已完成/未完成）
- ☐ 批量清除：点击"清除已完成"按钮一键删除所有已完成项
- ☐ 数据持久化：刷新页面后数据自动从localStorage恢复

□验收标准

- ☐ [] 能正常添加、删除、标记待办事项
- ☐ [] 筛选功能正确工作（全部/已完成/未完成）
- ☐ [] 刷新页面后数据保持（localStorage生效）
- ☐ [] 移动端显示正常（响应式布局）
- ☐ [] 有平滑的过渡动画效果
- ☐ [] 统计面板实时更新
- ☐ [] 代码符合Vue3 Composition API最佳实践

*□模块导航

√* Day 2 完成！你已掌握现代前端开发的核心技能！

明日将进入后端世界，打造完整的全栈能力！